

This project focuses on implementing an object detection model using YOLO11n (You Only Look Once) for efficient vehicle classification on CPU hardware.



Project Overview: Detecting Vehicles on CPU



Model: YOLO11n

The nano version of the YOLO architecture, optimized for speed.



Dataset: Custom

Curated vehicle dataset with specific classes: Bus, Car, and Truck.



Device: CPU

Training and inference performed on CPU, highlighting efficiency.



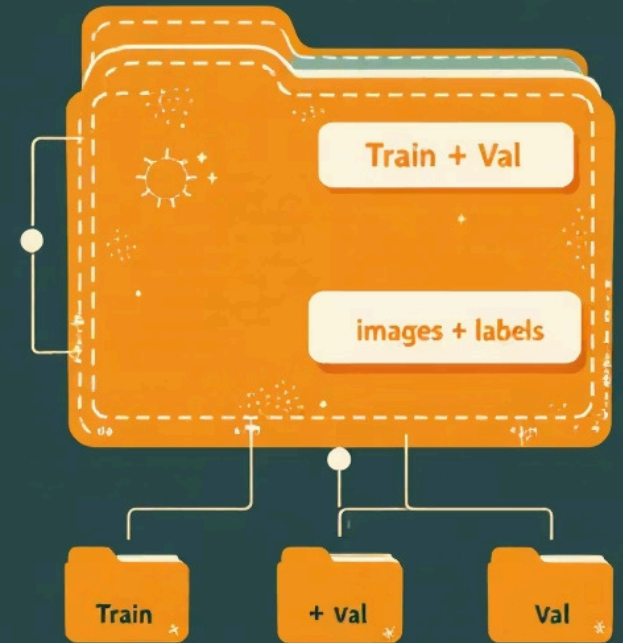
Training: 50 Epochs

Model trained for 50 epochs with 640x640 image size and batch size 4.

Dataset Structure: Organized for YOLO

The dataset is meticulously organized into `images` and `labels` directories, further subdivided into `train` and `val` sets, adhering strictly to the YOLO format. The `data.yaml` file configures the dataset.

```
datasets/  
├── data.yaml # Dataset configuration  
├── images/  
│   ├── train/ # Training images (80 images)  
│   └── val/ # Validation images (20 images)  
└── labels/  
    ├── train/ # Training labels  
    └── val/ # Validation labels
```



Core Project Files



The project is structured around key files that facilitate the entire object detection workflow, from model training to inference.

- **object_detection.ipynb**: Main Jupyter notebook with the complete workflow.
- **yolo11n.pt**: Pretrained YOLO11n model weights for initial setup.
- **datasets/**: Directory containing all structured dataset components.
- **runs/detect/train2/weights/best.pt**: Location of the best trained model weights after custom training.

Essential Libraries and Setup

1

Python 3.x

The foundational programming language for the project.

2

Ultralytics

Primary library for YOLO model implementation and training.

3

OpenCV

Used for image processing tasks within the workflow.

4

PyTorch

The deep learning framework powering the YOLO model.

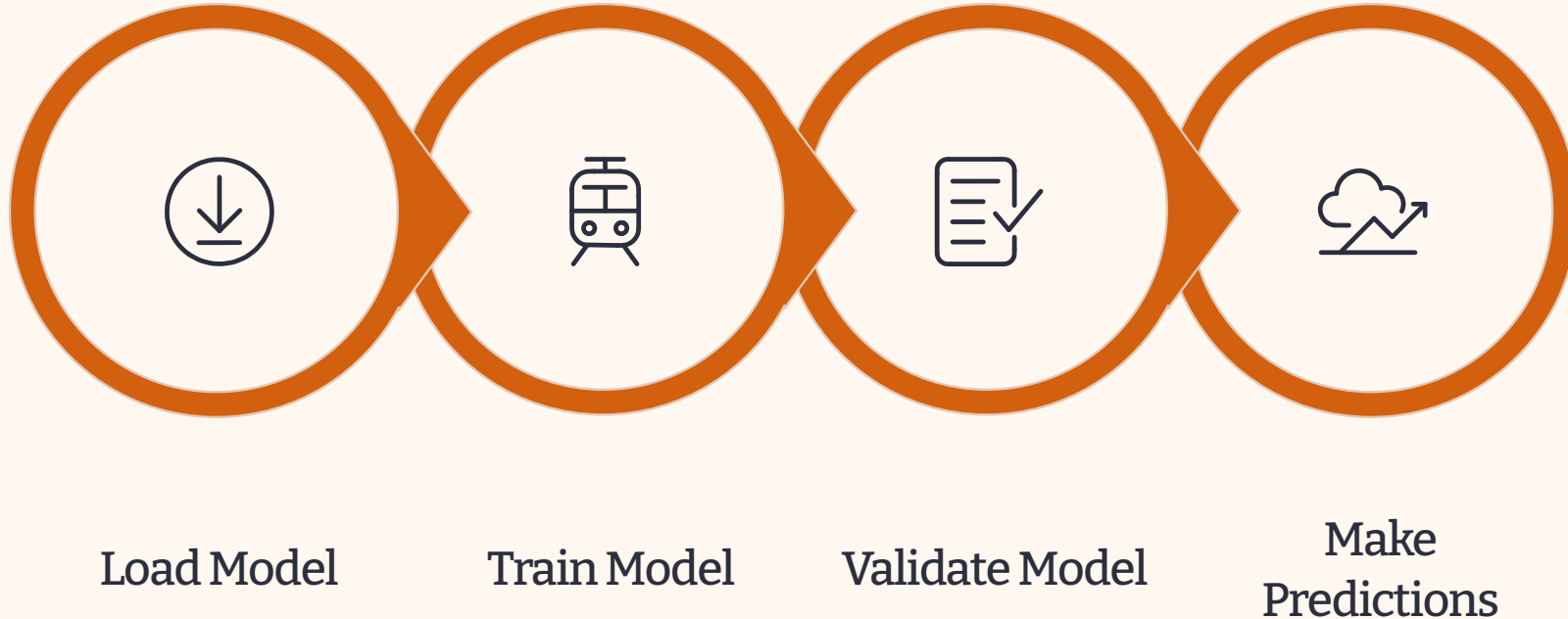
5

Matplotlib

For data visualization and plotting results.

Install all required dependencies using ``pip install ultralytics opencv-python matplotlib torch``.

Model Workflow: Training and Validation



This diagram illustrates the sequential steps involved in getting the YOLO model ready for vehicle detection, from initial loading to final prediction.

Code Snippets: Bringing YOLO to Life

1. Load Pretrained Model

```
from ultralytics import YOLO
pre_model = YOLO("yolo11n.pt")
```

2. Train the Model

```
model = YOLO("yolo11n.pt")
results = model.train(
    data=r"datasets/data.yaml",
    epochs=50,
    imgsz=640,
    batch=4,
    device="cpu"
)
```

3. Validate the Model

```
best_model = YOLO("runs/detect/train2/weights/best.pt")
best_model.val(data=r"datasets/data.yaml")
```

4. Make Predictions

```
img_path = r"datasets\images\val\17d0db0b-val_img_5.jpg"
results = best_model.predict(source=img_path, conf=0.5,
                             verbose=False)
```



Achieved Results

The trained model demonstrates enhanced detection performance compared to the pretrained version. It accurately identifies buses, cars, and trucks in images, showcasing its practical utility.

The training on a custom dataset has successfully adapted the YOLO11n model to the specific task of vehicle classification, proving its capability even on CPU-limited environments.

Potential Improvements for Enhanced Performance



Data Quality

Utilize higher resolution images and expand dataset diversity.



Hardware Upgrade

Transition to GPU for significantly faster training times.



Hyperparameter Tuning

Optimize parameters like learning rate and batch size for peak model performance.



Larger YOLO Models

Experiment with YOLO11m or YOLO11l for improved accuracy.



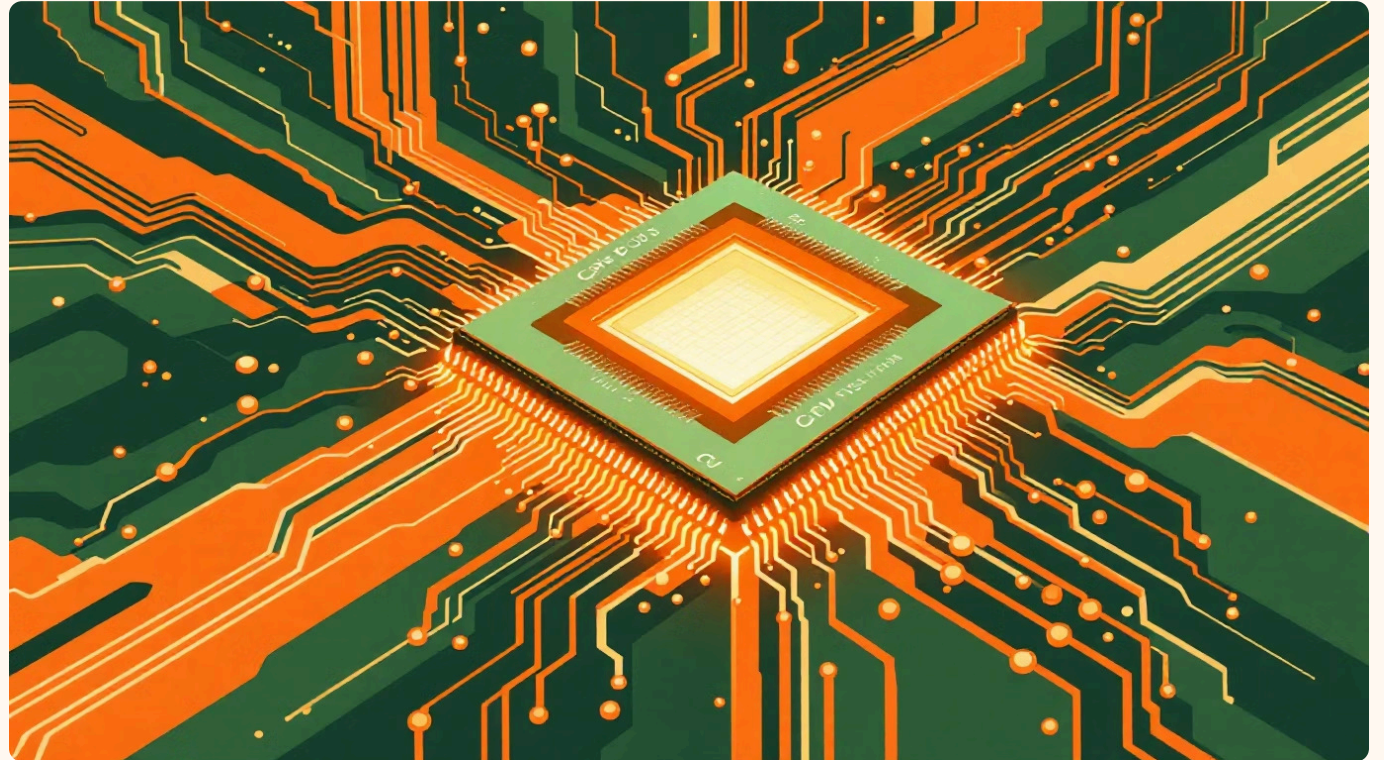
Data Augmentation

Implement advanced augmentation techniques to enrich the training data.

Key Takeaways & Project Notes

This project serves as an educational foundation for computer vision, specifically for vehicle detection using YOLO11n.

- Training on CPU extends the process duration compared to GPU, emphasizing resource constraints.
- A confidence threshold of 0.5 is applied for all predictions.
- Best performing model weights are saved at ``runs/detect/train2/weights/best.pt``.



COMPUTER VISION

OBJECT DETECTION

YOLO11N